# MONGODB

Sunita D. Rathod

# Overview

- MongoDB is an open-source, cross-platform, and distributed document-based database designed for ease of application development and scaling.
- It is a NoSQL database developed by MongoDB Inc
- MongoDB name is derived from the word "Humongous" which means huge, enormous.

# Overview

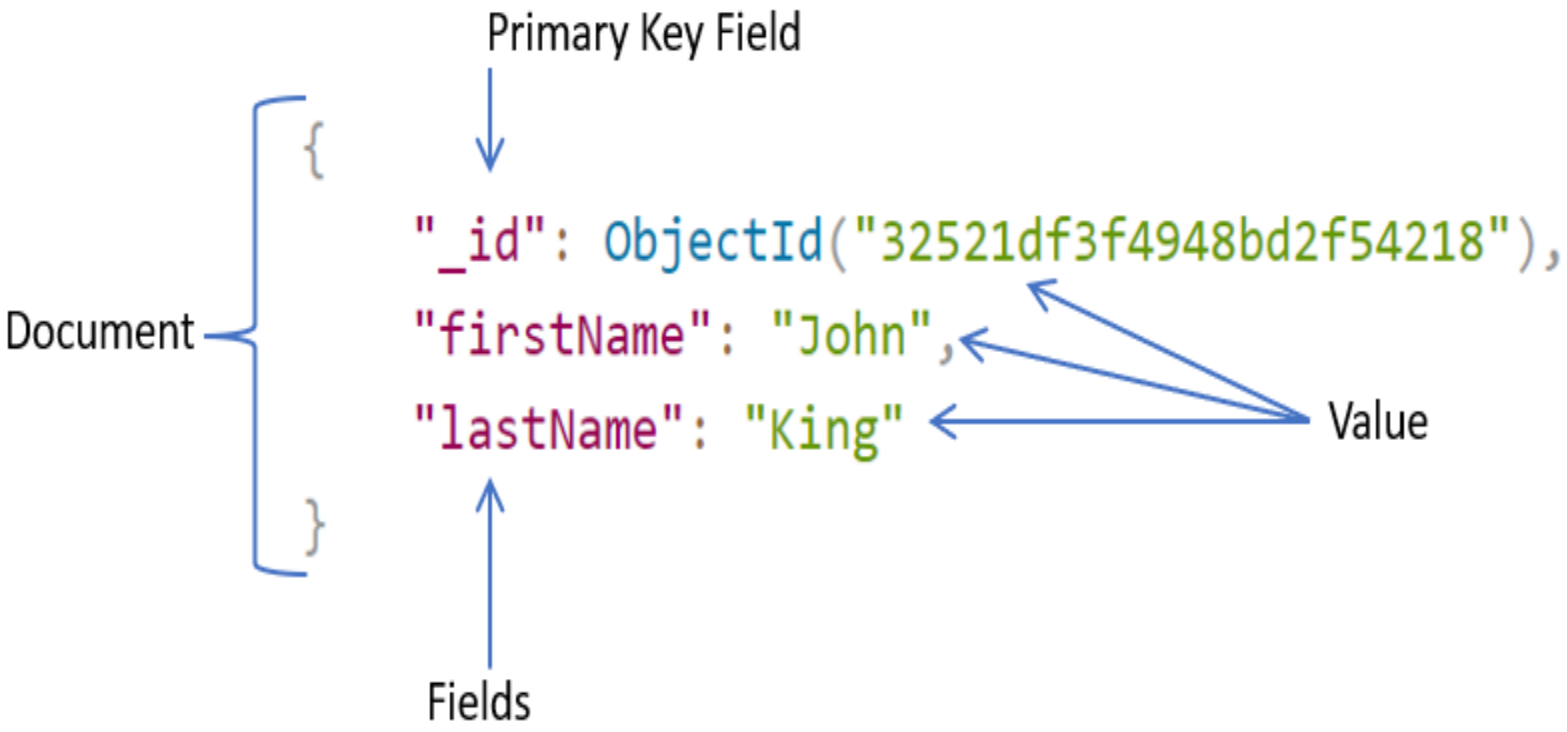
- MongoDB database is built to store a huge amount of data and also perform fast.
- MongoDB is not a Relational Database Management System (RDBMS). It's called a "NoSQL" database.
- It does not have tables, rows, and columns as other SQL (RDBMS) databases.
- It stores data in the collections as JSON based documents and does not enforce schemas.

# Relation between MongoDB and RDBMS terminologies

| MongoDB (NoSQL Database) | RDBMS (SQL Server, Oracle, etc.) |
|---|---|
| Database | Database |
| Collection | Table |
| Document | Row (Record) |
| Field | Column |

# Example of JSON based document

- Documents in a single collection can have different fields.



# Advantages of MongoDB

- MongoDB stores data as JSON based document that does not enforce the schema. It allows us to store hierarchical data in a document. This makes it easy to store and retrieve data in an efficient manner.

- It is easy to scale up or down as per the requirement since it is a document based database. MongoDB also allows us to split data across multiple servers.

- MongoDB provides rich features like indexing, aggregation, file store, etc.

# Advantages of MongoDB

- MongoDB performs fast with huge data.
- MongoDB provides drivers to store and fetch data from different applications developed in different technologies such as C#, Java, Python, Node.js, etc.
- MongoDB provides tools to manage MongoDB databases.

# Distinctive features of MongoDB

- Easy to use
- Light Weight
- Extremely faster than RDBMS

# Where MongoDB should be used

- Big and complex data
- Mobile and social infrastructure
- Content management and delivery
- User data management
- Data hub

# Performance analysis of MongoDB and RDBMS

- In relational database (RDBMS) tables are used as storing elements, while in MongoDB collection is used.
- In the RDBMS, we have multiple schema and in each schema we create tables to store data while, MongoDB is a document oriented database in which data is written in BSON format which is a JSON like format.
- MongoDB is almost 100 times faster than traditional database systems.

# MongoDB Datatypes

| Data Types | Description |
|---|---|
| String | String is the most commonly used datatype. It is used to store data. A string must be UTF 8 valid in mongodb. |
| Integer | Integer is used to store the numeric value. It can be 32 bit or 64 bit depending on the server you are using. |
| Boolean | This datatype is used to store boolean values. It just shows YES/NO values. |
| Double | Double datatype stores floating point values. |
| Min/Max Keys | This datatype compare a value against the lowest and highest bson elements. |

# MongoDB Datatypes

| Data Types | Description |
|---|---|
| Arrays | This datatype is used to store a list or multiple values into a single key. |
| Object | Object datatype is used for embedded documents. |
| Null | It is used to store null values. |
| Symbol | It is generally used for languages that use a specific type. |
| Date | This datatype stores the current date or time in unix time format. It makes you possible to specify your own date time by creating object of date and pass the value of date, month, year into it. |

# Create a new MongoDB Database

- You can use MongoDB_Shell or MongoDB Compass to create a new database.

- MongoDB Shell is the quickest way to connect, configure, query, and work with your MongoDB database.

- It acts as a command-line client of the MongoDB server.

# Create database

- There is no create database command in MongoDB.
- If there is no existing database, the following command is used to create a new database.
- Syntax:

```
use DATABASE_NAME
```

- If the database already exists, it will return the existing database. Else it will create one.

# check database

- To **check the currently selected database**, use the command db:

```
>db
```

- To **check the database list**, use the command show dbs:

```
>show dbs
```

# Drop Database

- The dropDatabase command is used to drop a database.
- It also deletes the associated data files.
- It operates on the current database.
- Syntax:

```
db.dropDatabase()
```

- This syntax will delete the selected database.

# Create Collection

- **Syntax:**

```
db.createCollection(name, options)
```

- **Name:** is a string type, specifies the name of the collection to be created.

- **Options:** is a document type, specifies the memory size and indexing of the collection. It is an optional parameter.

# list of options

| Field | Type | Description |
|---|---|---|
| Capped | Boolean | (Optional) If it is set to true, enables a capped collection. Capped collection is a fixed size collecction that automatically overwrites its oldest entries when it reaches its maximum size. If you specify true, you need to specify size parameter also. |
| AutoIndexID | Boolean | (Optional) If it is set to true, automatically create index on ID field. Its default value is false. |
| Size | Number | (Optional) It specifies a maximum size in bytes for a capped collection. Ifcapped is true, then you need to specify this field also. |
| Max | Number | (Optional) It specifies the maximum number of documents allowed in the capped collection. |

# example to create collection

```
>use test
```

```
switched to db test
```

```
>db.createCollection("SSSIT")
```

```
{ "ok" : 1 }
```

To **check the created collection**, use the command "show collections".

```
>show collections
```

```
SSSIT
```

# MongoDB create collection automatically

- MongoDB creates collections automatically when you insert some documents.

```
>db.SSSIT.insert({"name" : "seomount"})
>show collections
SSSIT
```

If you want to see the inserted document, use the find() command.

Syntax:

```
db.collection_name.find()
```

# Drop collection

- db.collection.drop() method is used to drop a collection from a database.

- Syntax:

```
db.COLLECTION_NAME.drop()
```

- Example:

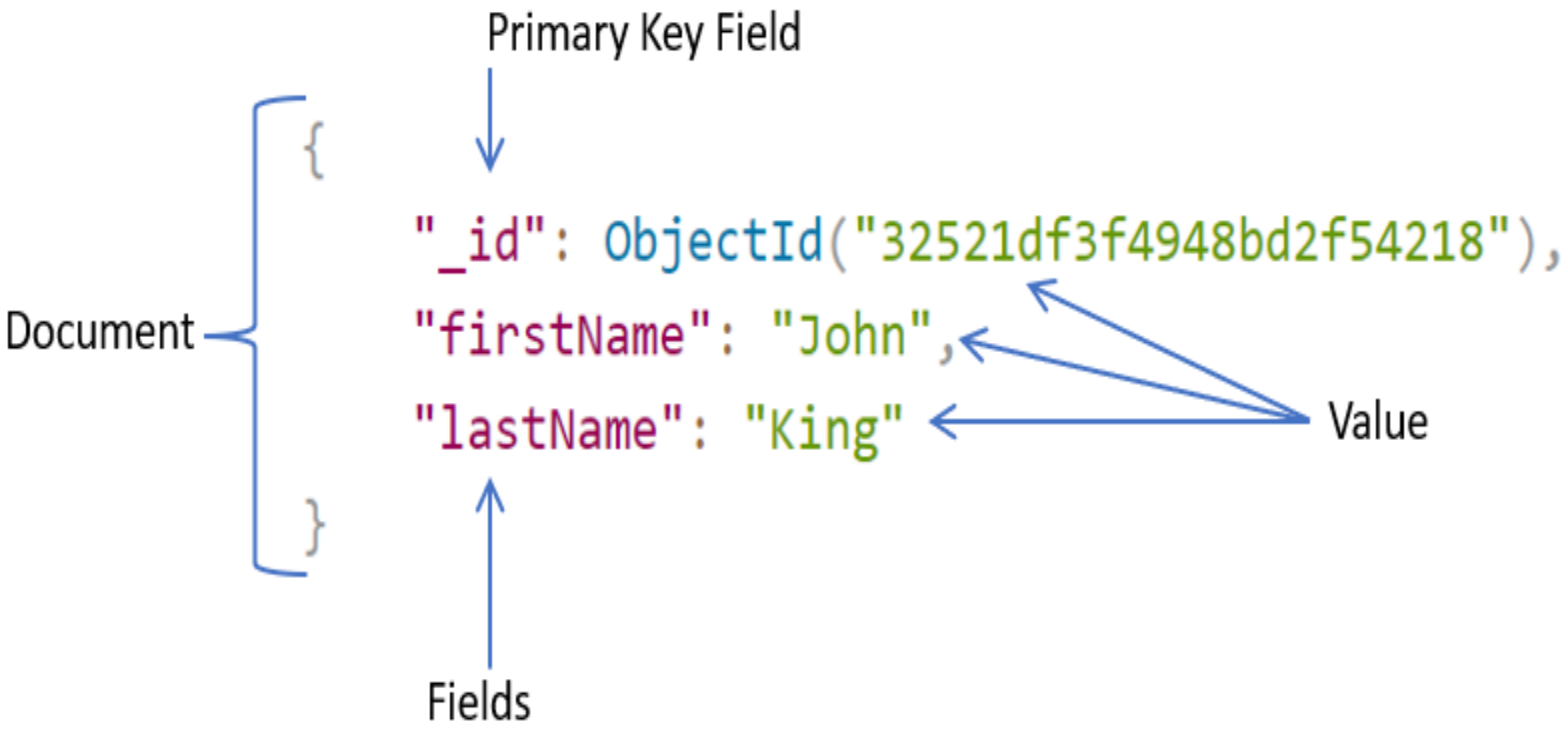
```
>db.SSSIT.drop()
True
```

# CRUD operations

- We can perform CRUD operations on documents.
- C – Create
- R – Read
- U – Update
- D – Delete

# Example of JSON based document

- Documents in a single collection can have different fields.



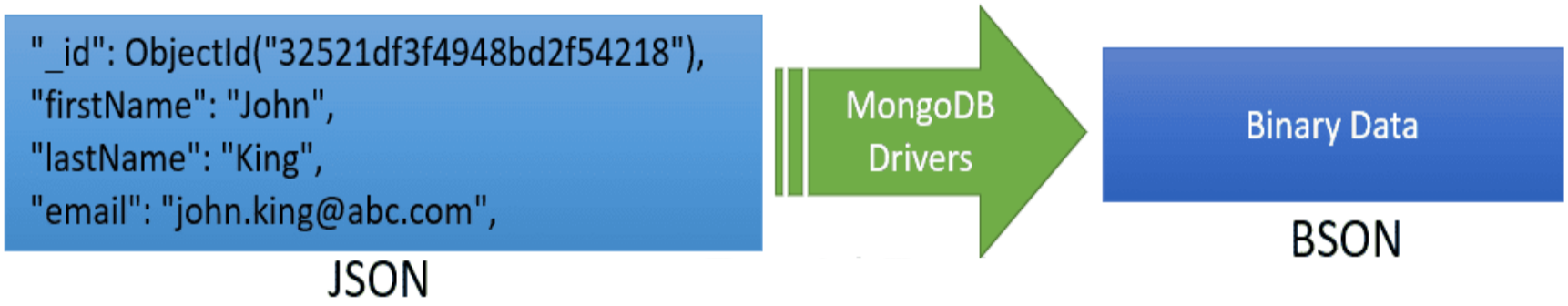
# Another Example:

Example: MongoDB Document

```
{
    _id: ObjectId("32521df3f4948bd2f54218"),
    firstName: "John",
    lastName: "King",
    email: "john.king@abc.com",
    salary: "33000",
    DoB: new Date('Mar 24, 2011'),
    skills: [ "Angular", "React", "MongoDB" ],
    address: {
                street:"Upper Street",
                house:"No 1",
                city:"New York",
                country:"USA"
            }
}
```

# JSON to BSON

- MongoDB stores data in key-value pairs as a BSON document.
- BSON is a binary representation of a JSON document that supports more data types than JSON.
- MongoDB drivers convert JSON document to BSON data.



# Embedded Documents:

- A document in MongoDB can have fields that hold another document. It is also called nested documents.

Example: Embedded Document

```
{
    _id: ObjectId("32521df3f4948bd2f54218"),
    firstName: "John",
    lastName: "King",
    department: {
                _id: ObjectId("55214df3f4948bd2f8753"),
                name:"Finance"
            },
    address: {
        phone: { type: "Home", number: "111-000-000" }
    }
}
```

# Embedded Documents:

- An embedded document can contain upto 100 levels of nesting.
- Supports a maximum size of 16 mb.
- Embedded documents can be accessed using dot notation embedded-document.fieldname,
- e.g. access phone number using address.phone.number.

# Array

- A field in a document can hold array. Arrays can hold any type of data or embedded documents.

Example: MongoDB Document with an Array

```
{
    _id: ObjectId("32521df3f4948bd2f54218"),
    firstName: "John",
    lastName: "King",
    email: "john.king@abc.com",
    skills: [ "Angular", "React", "MongoDB" ],
}
```

To specify or access the second element in the `skills` array, use `skills.1`.

# Insert documents

- MongoDB provides the following methods to insert documents into a collection:

- **insertOne()** - Inserts a single document into a collection.

- **insert()** - Inserts one or more documents into a collection.

- **insertMany()** - Insert multiple documents into a collection.

# insertOne()

Example: insertOne()

```
db.employees.insertOne({
    firstName: "John",
    lastName: "King",
    email: "john.king@abc.com"
})
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("616d44bea861820797edd9b0")
}
```

# find() & pretty()

- Use the **find()** to list all data of a collection, and the **pretty()** method to format resulted data.

```
db.employees.find().pretty()
```

Output

```
{
    _id: ObjectId("616d44bea861820797edd9b0"),
    firstName: "John",
    lastName: "King",
    email: "john.king@abc.com"
}
```

# insert()

Example: insert()

```
db.employees.insert({
    firstName: "John",
    lastName: "King",
    email: "john.king@abc.com"
})
```

Output

```
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("616d62d9a861820797edd9b2") }
}
```

## Example: Insert a Document

```
db.employees.insert(
    [
        {
            firstName: "John",
            lastName: "King",
            email: "john.king@abc.com"
        },
        {
                firstName: "Sachin",
                lastName: "T",
                email: "sachin.t@abc.com"
        },
        {
                firstName: "James",
                lastName: "Bond",
                email: "jamesb@abc.com"
        }
    ])
```

Output

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("616d63eda861820797edd9b3"),
    '1': ObjectId("616d63eda861820797edd9b4"),
    '2': ObjectId("616d63eda861820797edd9b5")
  }
}
```

# insertMany()

Example: Insert a Document

```
db.employees.insertMany(
    [
        {
            firstName: "John",
            lastName: "King",
            email: "john.king@abc.com"
        },
        {
                firstName: "Sachin",
                lastName: "T",
                email: "sachin.t@abc.com"
        },
        {
                firstName: "James",
                lastName: "Bond",
                email: "jamesb@abc.com"
        },
    ])
```

Output

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("616d63eda861820797edd9b3"),
    '1': ObjectId("616d63eda861820797edd9b4"),
    '2': ObjectId("616d63eda861820797edd9b5")
  }
}
```

## Example: insertMany() with Custom _id

```
db.employees.insertMany([
    {
        _id:1,
        firstName: "John",
        lastName: "King",
        email: "john.king@abc.com",
        salary: 5000
    },
    {
        _id:2,
        firstName: "Sachin",
        lastName: "T",
        email: "sachin.t@abc.com",
        salary: 8000
    },
    {
        _id:3,
        firstName: "James",
        lastName: "Bond",
        email: "jamesb@abc.com",
        salary: 7500
    },
```

```
    {
        _id:4,
        firstName: "Steve",
        lastName: "J",
        email: "steve.j@abc.com",
        salary: 9000

    },
    {
        _id:5,
        firstName: "Kapil",
        lastName: "D",
        email: "kapil.d@abc.com",
        salary: 4500

    },
    {
        _id:6,
        firstName: "Amitabh",
        lastName: "B",
        email: "amitabh.b@abc.com",
        salary: 11000
    }
])
```

Output

```
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5, '5': 6 }
}
```

# Finding documents

- MongoDB provides two methods for finding documents from a collection:

- **findOne()** - returns a the first document that matched with the specified criteria.

- **find()** - returns a cursor to the selected documents that matched with the specified criteria.

# findOne()

- The **findOne()** method returns the first document that is matched with the specified criteria.

Syntax:

```
db.collection.findOne(query, projection)
```

Parameters:

1. query: Optional. Specifies the criteria using query operators.
2. projection: Optional. Specifies the fields to include in a resulted document.

## Example: find()

```
db.employees.findOne()
```

## Output

```
{
  _id: 1,
  firstName: 'John',
  lastName: 'King',
  email: 'john.king@abc.com',
  salary: 5000
}
```

Example: find()

```
db.employees.findOne({firstName: "Kapil"})
```

Output

```
{
  _id: 5,
  firstName: 'Kapil',
  lastName: 'D',
  email: 'kapil.d@abc.com',
  salary: 4500
}
```

The `findOne()` method performs the **case-sensitive** search, so `{firstName: "Kapil"}` and `{firstName: "kapil"}` returns different result.

# Example

- Use the query operators for more refined search.

Example: find()

```
db.employees.findOne({salary: {$gt: 8000}})
```

Output

```
{
  _id: 4,
  firstName: 'Steve',
  lastName: 'J',
  email: 'steve.j@abc.com',
  salary: 9000
}
```

# Projection

Example: find()

```
db.employees.findOne({firstName: "Sachin"}, {firstName:1, lastName:1})
```

Output

```
{ _id: 2, firstName: 'Sachin', lastName: 'T' }
```

# Update documents

- MongoDB provides the following methods to update existing documents in a collection:

- **db.collection.updateOne()** - Modifies a single document in a collection.

- **db.collection.updateMany()** - Modifies one or more documents in a collection.

# updateOne()

Syntax:

```
db.collection.updateOne(filter, document, options)
```

Parameters:

1. filter: The selection criteria for the update, same as find() method.
2. document: A document or pipeline that contains modifications to apply.
3. options: Optional. May contains options for update behavior. It includes upsert, writeConcern, collation, etc.

## Example: updateOne()

```
db.employees.updateOne({_id:1}, { $set: {firstName:'Morgan'}})
```

## Output

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

## Check Updated Document

```
db.employees.find({_id:1})
```

## Output

```
{
  _id: 1,
  firstName: 'Morgan',
  lastName: 'King',
  email: 'john.king@abc.com',
  salary: 5000
}
```

# Update Multiple Fields

Example: Update Multiple Fields

```
db.employees.updateOne({_id:2}, { $set: {lastName:"Tendulkar", email:"sachin.tendulkar@abc.com"}})
```

Output

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

## Check Updated Document

```
db.employees.find({_id:2})
```

## Output

```
{
    _id:2,
    firstName: "Sachin",
    lastName: "Tendulkar",
    email: "sachin.tendulkar@abc.com",
    salary: 8000
}
```

# Upsert - Add if not Exist

Example: Upsert

```
db.employees.updateOne({firstName:"Heer"}, { $set: {lastName:"Patel", salary:2000}})
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("6172a2e9ce7d5ca09d6ab082"),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
```

MongoDB adds a new document with new `_id`, because it cannot find a document with the `firstName:"Heer"`.

# Update Operators

| Method | Description |
| --- | --- |
| $currentDate | Sets the value of a field to current date, either as a Date or a Timestamp. |
| $inc | Increments the value of the field by the specified amount. |
| $min | Only updates the field if the specified value is less than the existing field value. |
| $max | Only updates the field if the specified value is greater than the existing field value. |
| $mul | Multiplies the value of the field by the specified amount. |
| $rename | Renames a field. |
| $set | Sets the value of a field in a document. |
| $setOnInsert | Sets the value of a field if an update results in an insert of a document. Has no effect on update operations that modify existing documents. |
| $unset | Removes the specified field from a document. |

# updateMany()

Example: updateMany()

```
db.employees.updateMany({ salary:7000 }, { $set: { salary:8500 }})
```

Output

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

- If you specify an empty filter criteria {}, then it will update all the documents.

Example: updateMany()

```
db.employees.updateMany({}, { $set: {location: "USA"}})
```

Output

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 6,
  modifiedCount: 6,
  upsertedCount: 0
}
```

# Add New Element to Arrays

- Use the **$push** array operator to add new elements to arrays.

Example: Add Array Elements

```
db.employees.updateMany(
    {},
    {$push:{"skills":"Sports"}}) // add "Sports" to all arrays

db.employees.updateMany(
    {_id:3},
    {$push:{"skills":"Sports"}}) // add "Sports" element to skills array where _id:3
```

# Remove First or Last Element from Arrays

- Use the **$pop** operator to remove first or last element from arrays. Specify 1 to remove the last element and -1 to remove the first element.

Example: Delete Array Element

```
db.employees.updateMany(
    {},
    {$pop:{"skills":1}}) // removes the last element

db.employees.updateMany(
    {},
    {$pop:{"skills":-1}}) //removes the first element

db.employees.updateMany(
    {},
    {$pull: { "skills": "GST" }}) // removes "GST"
```

# Delete Documents

- MongoDB provides the following methods to delete one or more documents in a collection.

- **db.collection.deleteOne()** - Deletes the first matching document in a collection.

- **db.collection.deleteMany()** - Deletes all the matching documents in a collection.

# deleteOne()

Example: Delete Single Document using deleteOne()

```
db.employees.deleteOne({ salary:7000 })
```

Output

```
{ acknowledged: true, deletedCount: 1 }
```

# deleteMany()

Example: deleteMany()

```
db.employees.deleteMany({ salary:7000 })
```

Output

```
{ acknowledged: true, deletedCount: 2 }
```

The following deletes all documents where `salary` is less than `7000`.

Example: deleteMany()

```
db.employees.deleteMany({ salary: { $lt:7000} })
```

Output

```
{ acknowledged: true, deletedCount: 2 }
```

# deleteMany()

Example: Delete All Documents

```
db.employees.deleteMany({ }) //deletes all documents
```

Output

```
{ acknowledged: true, deletedCount: 6 }
```

# Different Types of MongoDB Operators

- 1) Comparison Operators
- 2) Logical Operators
- 3) Array Operators
- 4) Element Operators

# Comparison Operators

| Name | Description |
|---|---|
| **$eq** | Matches values that are equal to the value specified in the query |
| **$ne** | Matches all values that are not equal to the value specified in the query. |
| **$gt** | Matches values that are greater than the value specified in the query. |
| **$gte** | Matches values that are greater than or equal to the value specified in the query. |
| **$lt** | Matches values that are less than the value specified in the query. |
| **$lte** | Matches values that are less than or equal to the value specified in the query. |
| **$in** | Matches any of the values that exist in an array specified in the query. |
| **$nin** | Matches values that do not exist in an array specified in the query. |

# Logical Operators

| Name | Description |
| --- | --- |
| **$and** | Returns all documents that match the conditions of both expressions. |
| **$or** | Returns all documents that match the conditions of either expression. |
| **$nor** | Returns all documents that **do not** match the conditions of either expression |
| **$not** | Inverts the effect of a query expression and returns documents that **do *not*** match the query expression. |

# Array Operators

| Name | Description |
| --- | --- |
| **$all** | Returns documents from a collection if it matches all values in the specified array. |
| **$size** | Returns documents from the collection to match an array with the provided number of elements in it. |
| **$elemMatch** | Returns documents if they Match more than one component (all specified $elemMatch conditions) within an array element. |

# Element Operators

| Name | Description |
| --- | --- |
| **$exists** | Returns documents that have a specific field. |
| **$type** | Returns documents if field is of a specified type. |

# Relations in MongoDB

- In MongoDB, one-to-one, one-to-many, and many-to-many relations can be implemented in two ways:

- Using embedded documents

- Using the reference of documents of another collection

## Example: Relation using Embedded Document

```
db.employee.insertOne({
      _id: ObjectId("32521df3f4948bd2f54218"),
     firstName: "John",
     lastName: "King",
     email: "john.king@abc.com",
     salary: "33000",
     DoB: new Date('Mar 24, 2011'),
     address: {
                  street:"Upper Street",
                  house:"No 1",
                  city:"New York",
                  country:"USA"
               }
})
```

## Example: Implement One-to-One Relation using Reference

```
db.address.insertOne({
     _id: 101,
    street:"Upper Street",
    house:"No 1",
    city:"New York",
    country:"USA"
})

db.employee.insertOne({
    firstName: "John",
    lastName: "King",
    email: "john.king@abc.com",
    salary: "33000",
    DoB: new Date('Mar 24, 2011'),
    address: 101
})
```

# Aggregation Pipelines

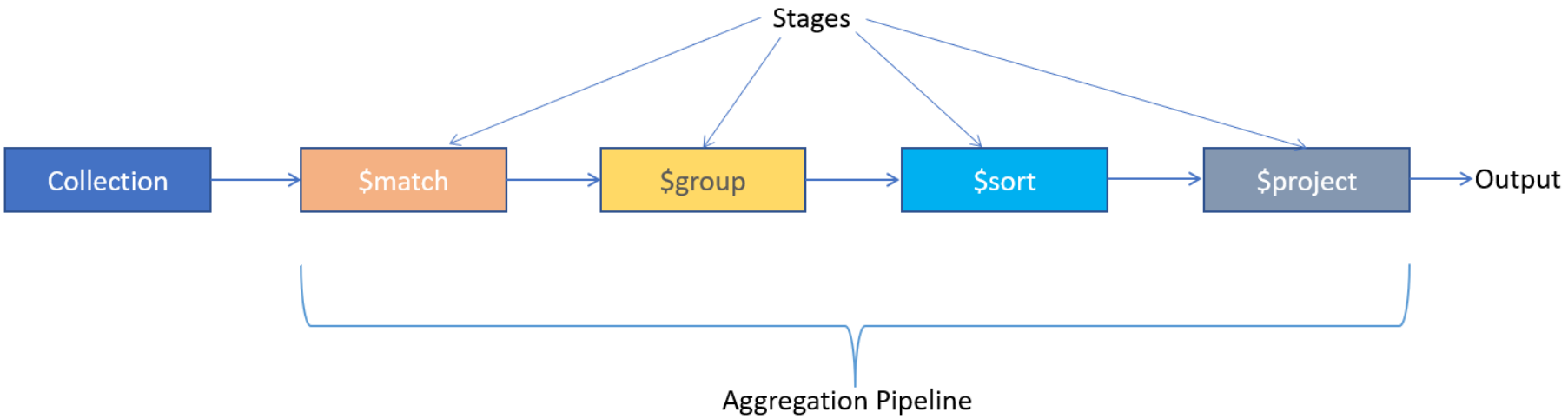
- The aggregation pipeline is an array of one or more stages passed in the db.aggregate() or db.collection.aggregate() method

```
db.collection.aggregate([ {stage1}, {stage2}, {stage3}...])
```

# Aggregation Pipelines

- Every stage receives the output of the previous stage, processes the data further, and sends it to the next stage as input data.

- Aggregation pipeline executes on the server can take advantage of indexes.



# Example

Example: Sort Groupped Data

```
db.employees.aggregate([
    { $match:{ gender:'male'}},
    { $group:{ _id:{ deptName:'$department.name'}, totalEmployees: { $sum:1} } },
    { $sort:{ deptName:1}}
])
```

Output

```
[
  { _id: { deptName: 'Finance' }, totalEmployees: 2 },
  { _id: { deptName: 'HR' }, totalEmployees: 1 },
  { _id: { deptName: 'Marketing' }, totalEmployees: 2 }
]
```